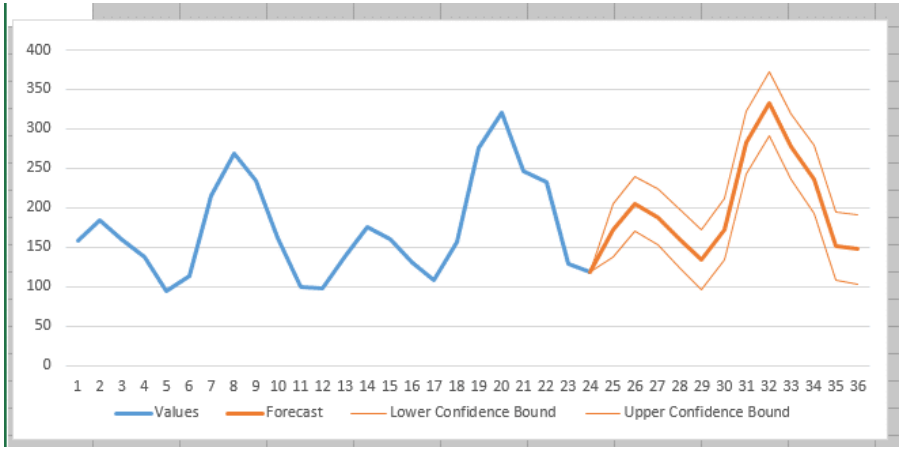
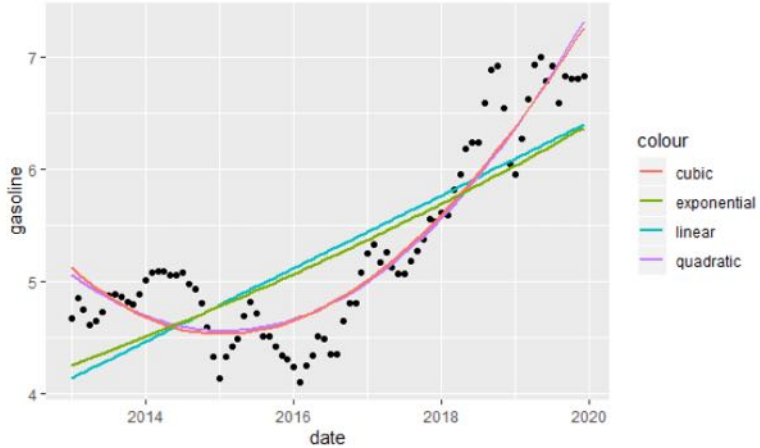
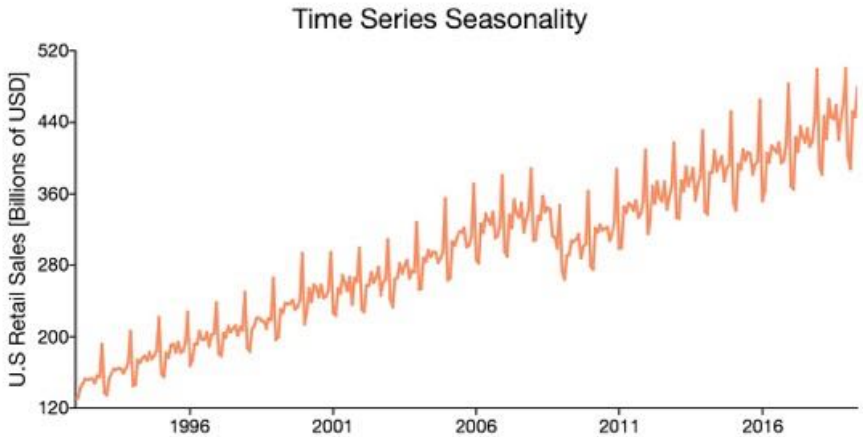
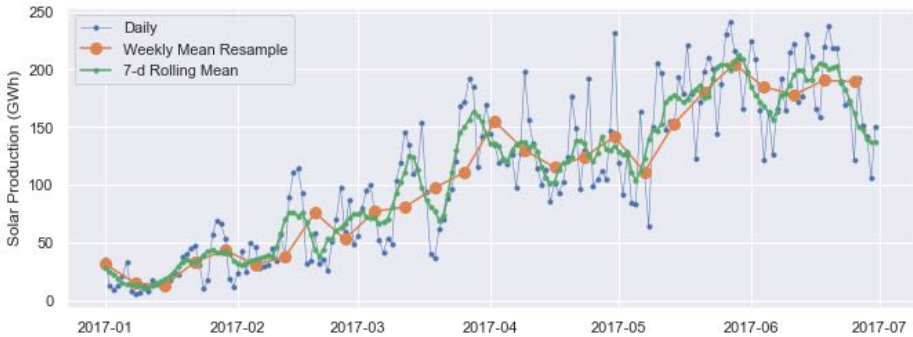
Time Series Forecasting









Forecasting methods

- Qualitative, Naive (forecast = last value, or an average of last period)
- **Trend, Linear Regression**
- **Multivariate Linear Regression**
- **Exponential Smoothing** (weighted moving average)
- **Holt-Winters** = Triple Exponential Smoothing (smooth, trend, seasonality)
- **Seasonality** in the model using Fourier decomposition (over 4+ years)
- **ARMA** (Auto-Regressive Moving Average), **ARIMA** (Auto-Regressive Integrated Moving Average)
  SARIMA (S=Seasonality), **SARIMAX** (X - eXogenous regressors)
- Auto-correlations, bivariate cross-correlation, multivariate time series analysis
- **Causal** (cause-effect) - modeling specific causes/drivers that can influence the target value
  (drivers – number of resources, spending on advertising, promotion, legal change,
  website redesign, sport event, new law, election, etc.).
  Some drivers cause permanent shift, others – temporary response.
- **Random Walk Simulation (Monte-Carlo)** – when having causal model
- **Classification** (Random Forest, Boosting, etc.) - "this type of ... historically was causing ..."
- **Neural Networks** - LSTM (Long Short Term Memory), multivariate LSTM, GRU (Gated Recurrent Unit)
- **Facebook Prohpet** -
- **Azure AutoML** -
- etc.

Forecasting in Power BI

Holt-Winters' multiplicative method (~ 1960)

$$\hat{y}_{t+h|t} = (\ell_t + hb_t)s_{t+h-m(k+1)}$$
$$\ell_t = \alpha \frac{y_t}{s_{t-m}} + (1-\alpha)(\ell_{t-1} + b_{t-1})$$
$$b_t = \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)b_{t-1}$$
$$s_t = \gamma \frac{y_t}{(\ell_{t-1} + b_{t-1})} + (1-\gamma)s_{t-m}$$

Power BI uses exponential smoothing and backtesting (hindcasting). There are two algorithms:

.... a variation of the Holt-Winters algorithm for the seasonal data with a few changes to deal with noisy data

.... a simpler model for non-seasonal data which has a term for additive trend and additive error

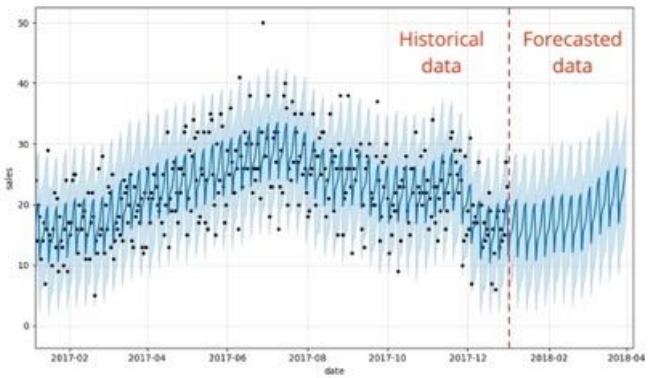
For the seasonal forecasting, PBI modify the Holt-Winters algorithm by computing the sum of squares of prediction errors for what is called a validation window, instead of using the one-step ahead.

Another modification they make is when they detect large variations, they adjust the trend of the time window to a more recent to capture the overall trend of the time series.

... https://otexts.com/fpp2/holt-winters.html -
... https://powerbi.microsoft.com/es-es/blog/describing-the-forecasting-models-in-power-view/#ETSAAA -
... https://powerbi.microsoft.com/en-us/blog/introducing-new-forecasting-capabilities-in-power-view-for-office-365/#:~:text=The%20Power%20BI%20Team&text=Forecasting%20in%20Power%20View%20utilizes,from%20a%20series%20of%20data -
... https://stoneridgesoftware.com/take-advantage-of-forecasting-in-power-bi-with-the-analytics-pane/ -
... https://pawarbi.github.io/blog/forecasting/python/powerbi/forecasting_in_powerbi/2020/04/24/timeseries-powerbi.html -
... https://pawarbi.github.io/blog/forecasting/python/powerbi/forecasting_in_powerbi/2020/04/24/timeseries-powerbi.html#ETS(AAN) -

Forecasting using Facebook Prophet

.. https://github.com/facebook/prophet -
.. https://youtu.be/pOYAXv15r3A - (video)
.. https://facebook.github.io/prophet/docs/quick_start.html -
.. https://databricks.com/blog/2020/01/27/time-series-forecasting-prophet-spark.html -
.. Forecasting time series data based on an additive model where non-linear trends are fit with yearly, weekly, and daily seasonality, plus holiday effects.
.. It works best with time series that have strong seasonal effects and several seasons of historical data.
.. Prophet is robust to missing data and shifts in the trend, and typically handles outliers well.



black dots - actuals
dark blue - predictions
light blue - 95% uncertainty intervals

The input to Prophet is always a dataframe with two columns: **ds** and **y**.

The **ds** (datestamp) column should be of a format expected by Pandas, ideally **YYYY-MM-DD** for a date or **YYYY-MM-DD HH:MM:SS** for a timestamp.

The **y** column must be numeric, and represents the measurement we wish to forecast.

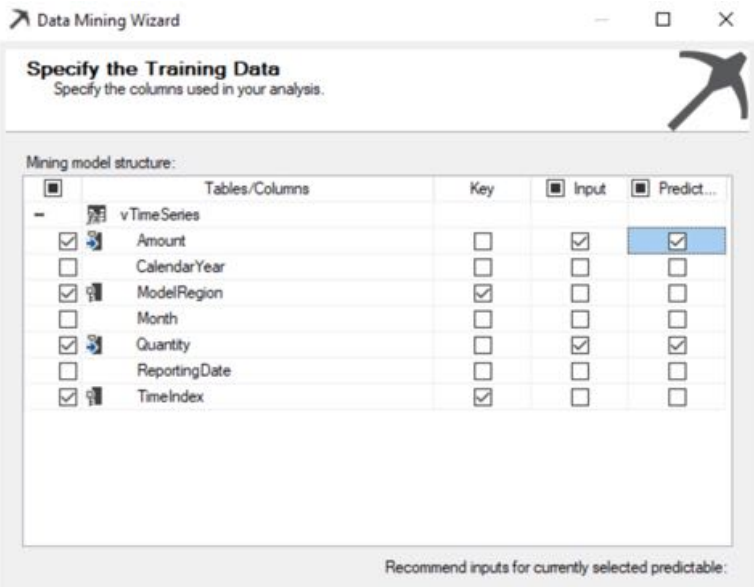
Neural Prophet

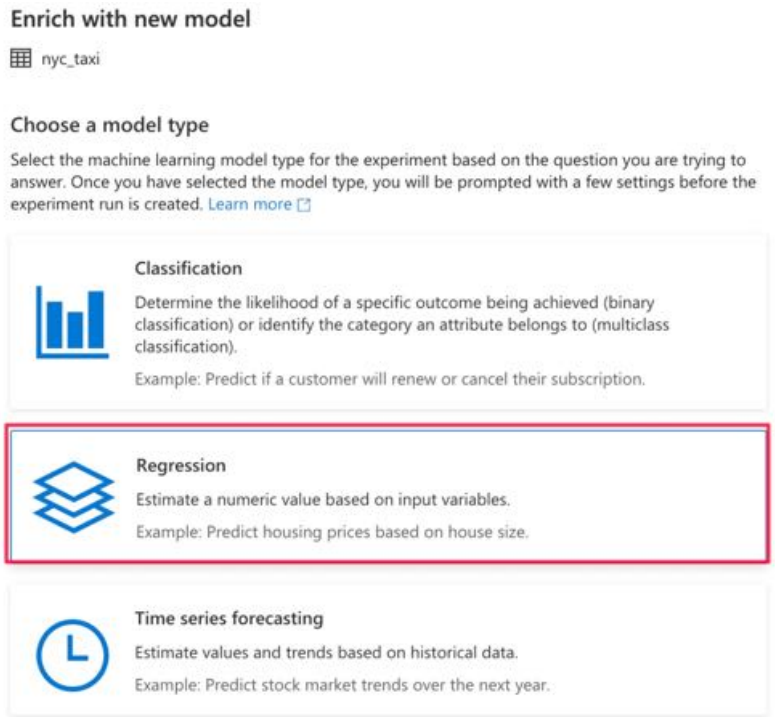
.. https://towardsdatascience.com/neural-prophet-a-time-series-modeling-library-based-on-neural-networks-dd02dc8d868d -
.. https://github.com/ourownstory/neural_prophet -
.. Much faster than Prophet, uses PyTorch - https://pytorch.org/ , uses AR-Net for modeling time-series autocorrelation
.. https://ai.facebook.com/blog/ar-net-a-simple-autoregressive-neural-network-for-time-series/ -
.. Custom losses and metrics, configurable non-linear layers of feed-forward neural networks, etc.

Forecasting Using Azure AutoML or MSFT Time Series in SQL Server

AutoML Wizard - https://docs.microsoft.com/en-us/azure/synapse-analytics/machine-learning/tutorial-automl

Microsoft Time Series in SQL Server
- https://www.sqlshack.com/microsoft-time-series-in-sql-server/





Model Quality Evaluation

How to Calculate Forecast Accuracy:

https://www.callcentrehelper.com/calculate-forecast-accuracy-113808.htm

There are multiple methods:
MAPE = Mean Absolute Percentage Error
WAPE = Weighted Absolute Percentage Error
MSE = Mean Squared Error
RMSE = Root Mean Square Error
NMAPE – Normalized MAPE
NRMSE – Normalized RMSE
R2 score
Spearman Correlation
etc.

Normalization can be done in different ways:

- by difference between maximum and minimum
- by standard deviation of the "actual" curve
- by mean value - not good, error will depend on vertical shift
- by interquartile range (i.e. the difference between 25th and 75th percentile) – not good, depends ondistribution of points between quartiles

Real life error function should be custom fit to your data and business case.
And normalization should not depend only on min/max or mean – but probably on combination of different characteristics – and treat "zero" cases carefully.

$$\frac{1}{N}\sum_{t=1}^{N}\frac{ABS(Actual_t - Forecast_t)}{Actual_t} * 100\%$$

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\frac{|Y_i - \widehat{Y_i}|}{Y_i}$$

$$\text{WAPE} = \frac{\sum_{i,t}|y_{i,t} - \hat{y}_{i,t}|}{\sum_{i,t}|y_{i,t}|}$$

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \widehat{Y_i})^2$$

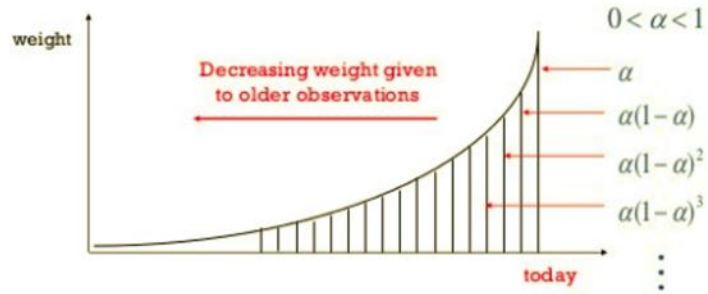
More Details about different forecasting models

Exponential Smoothing

Exponential Smoothing - a weighted moving average, where weights decline exponentially for older events.
If x(t) is real time series, then we construct forecast f(t) step by step using formula:

$$f_{t+1} = a*x_t + (1-a)*f_t$$

We initiate f_0 as an average of first several points.
For future points (once we out of historical period) we simply extrapolate last value into the future.



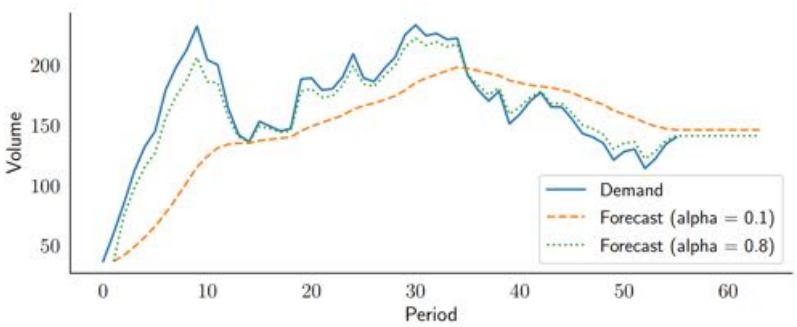


Figure 3.2: Simple smoothing

Limitations of simple exponential smoothing model:

- it does not project trends
- it does not recognize seasonal patterns
- it cannot use any external information

Good simple explanations by Nicolas Vandeput:
.. https://towardsdatascience.com/simple-exponential-smoothing-749fc5631bed
Python code:
.. https://towardsdatascience.com/diy-simple-exponential-smoothing-with-python-dbb570d30fb5

Holt-Winters = Triple Exponential Smoothing

Holt-Winters = Triple Exponential Smoothing

1st smoothing (level) -exponential smoothing for value
2nd smoothing (trend) - exponential smoothing for slope
3rd smoothing - exponential smoothing comparing to previous cycle

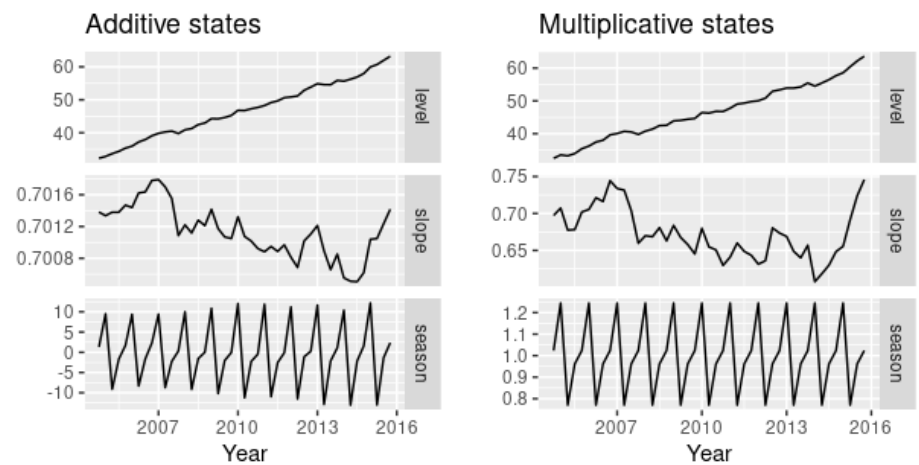
.. https://towardsdatascience.com/holt-winters-exponential-smoothing-d703072c0572 -
.. https://orangematter.solarwinds.com/2019/12/15/holt-winters-forecasting-simplified/ -

additive method

$$\hat{y}_{t+h|t} = \ell_t + hb_t + s_{t+h-m(k+1)}$$
$$\ell_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1})$$
$$b_t = \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)b_{t-1}$$
$$s_t = \gamma(y_t - \ell_{t-1} - b_{t-1}) + (1-\gamma)s_{t-m},$$

multiplicative method

$$\hat{y}_{t+h|t} = (\ell_t + hb_t)s_{t+h-m(k+1)}$$
$$\ell_t = \alpha\frac{y_t}{s_{t-m}} + (1-\alpha)(\ell_{t-1} + b_{t-1})$$
$$b_t = \beta^*(\ell_t - \ell_{t-1}) + (1-\beta^*)b_{t-1}$$
$$s_t = \gamma\frac{y_t}{(\ell_{t-1} + b_{t-1})} + (1-\gamma)s_{t-m}$$



The **additive method** is preferred when the seasonal variations are roughly constant through the series, while the multiplicative method is preferred when the seasonal variations are changing proportional to the level of the series.

With the additive method, the seasonal component is expressed in absolute terms in the scale of the observed series, and in the level equation the series is seasonally adjusted by subtracting the seasonal component.

Within each year, the seasonal component will add up to approximately zero.

With the **multiplicative method**, the seasonal component is expressed in relative terms (percentages), and the series is seasonally adjusted by dividing through by the seasonal component. Within each year, the seasonal component will sum up to approximately m .

ARMA, ARIMA, SARIMAX

Auto-Regressive Moving Average model
Auto-Regressive Integrated Moving Average model
Seasonal Autoregressive Integrated Moving Average Exogenous model

- https://en.wikipedia.org/wiki/Autoregressive%E2%80%93moving-average_model
- https://en.wikipedia.org/wiki/Autoregressive_integrated_moving_average
- https://www.statsmodels.org/dev/examples/notebooks/generated/statespace_sarimax_stata.html
- https://365datascience.com/tutorials/python-tutorials/sarimax/

Autoregressive Model:
- https://www.youtube.com/watch?v=5-2C4eO4cPQ -
Autocorrelation and Partial Autocorrelation
- https://www.youtube.com/watch?v=DeORzP0go5I -

ACF and PACF:
- https://towardsdatascience.com/significance-of-acf-and-pacf-plots-in-time-series-analysis-2fa11a5d10a8 -

ARIMA(p,d,q) (Auto-Regressive Integrated Moving Average)
p - number of autoregressive terms,
d - number of nonseasonal differences needed for stationarity
q - number of lagged forecast errors
ARIMA(1,0,0) - 1st order autoregressive model
ARIMA(2,0,0)) - oscillations
ARIMA(0,1,0) - random walk
ARIMA(1,1,0) - differenced first-order autoregressive model
ARIMA(0,1,1) - simple exponential smoothing:
ARIMA(0,1,1) - simple exponential smoothing with growth
ARIMA(1,1,1) - ...
ARIMA(0,2,1) - linear exponential smoothing
ARIMA(0,2,2) - linear exponential smoothing

ARMA (Auto-Regressive Moving Average) - tries to describe a process as a sum of two processes - signal (AR) and noise (MA) - and then extrapolate signal into the future.
.. https://people.duke.edu/~rnau/411arim.htm

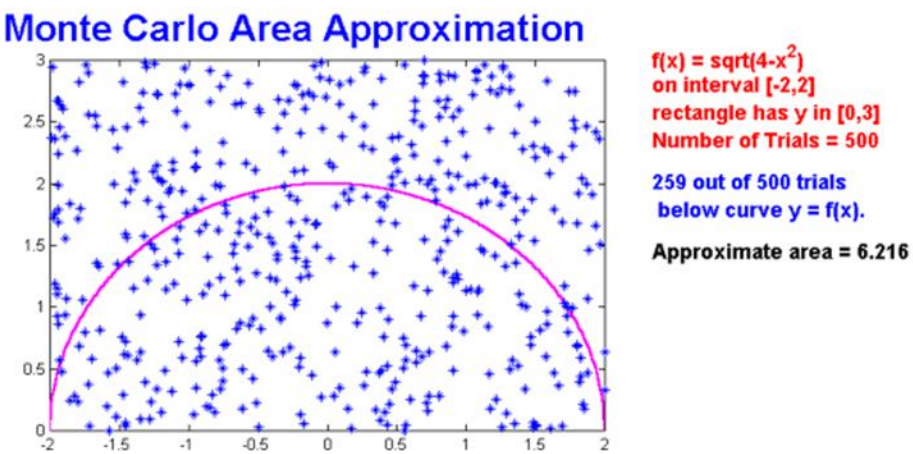
ARMA(t) = AR(t) + MA(t), where AR is a weighted sum of "P" previous **values**, and MA is weighted average of "Q" previous (white noise) **errors**:
AR : Y(t) = a0 + a1*Y(t-1) + ... + aP*Y(t-P) +
Z(t) -b1*Z(t-1) - ... -bQ*Z(t-Q)

SARIMAX

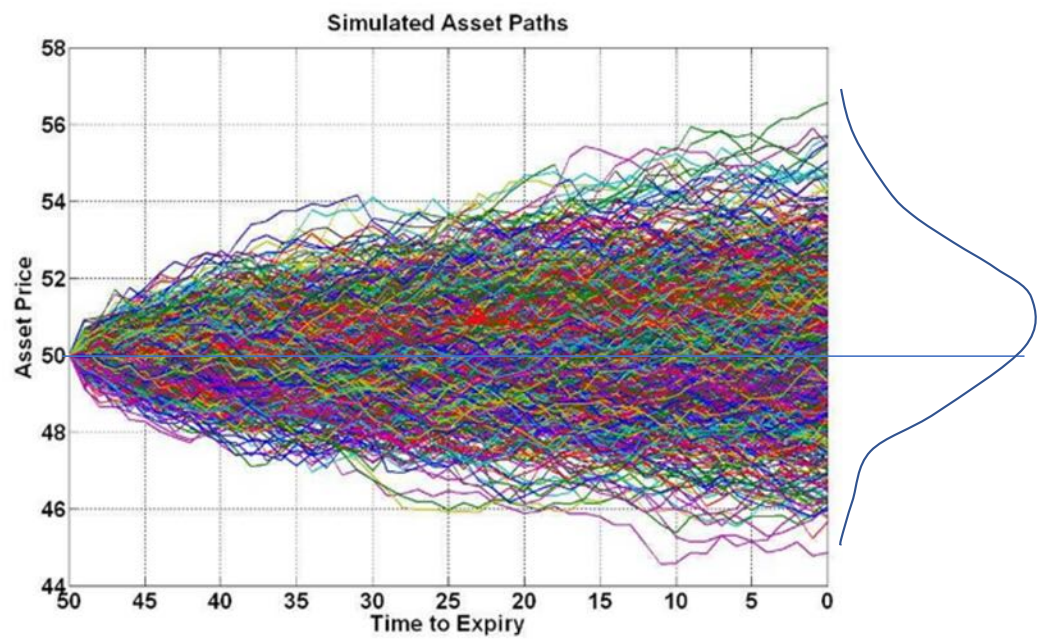
SARIMAX (1, 0, 2) (2, 0, 1, 5)

$$y_t = c + \varphi_1 y_{t-1} + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \Phi_1 (y_{t-5} + \varphi_1 y_{t-6}) + \Phi_2 (y_{t-10} + \varphi_1 y_{t-11}) + \Theta_1 (\varepsilon_{t-5} + \theta_1 \varepsilon_{t-6} + \theta_2 \varepsilon_{t-7}) + \varepsilon_t$$

Monte Carlo Examples:



In the example above relative error is ~1/sqrt(N)
= 1/sqrt(259.0) = 0.062 (or ~6%)



MCMC – Markov Chain Monte Carlo (random walk)

What is “Monte Carlo” ?

Stanislaw Ulam John von Neuman Nicholas Metropolis

Monte Carlo method - use random sampling to obtain numerical results. It was invented in the late 1940s by **Stanislaw Ulam**, while he was working on nuclear weapons projects at the Los Alamos National Laboratory.

Nicholas Metropolis has suggested using the name **Monte Carlo**, which refers to the **Monte Carlo** Casino in Monaco.

Monte Carlo officially refers to an administrative area of the Principality of Monaco, specifically the ward of Monte Carlo/Spélugues, where the **Monte Carlo** Casino is located.

Casino’s name “**Monte Carlo**” means literally "**Mount Charles**”. It was chosen in 1863 in honor of Charles III, Prince of Monaco.

Time series forecasting with Multiple Linear Regression

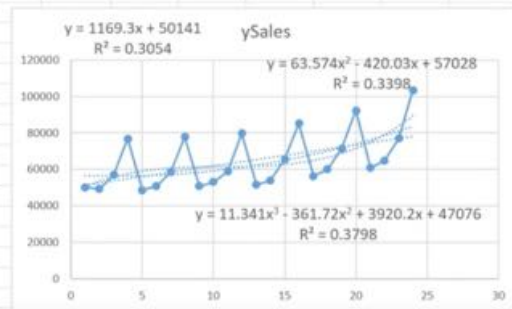
Use multi-dimensional linear regression to get coefficients of the model.
.. Take quarterly values (or monthly values) – and put them into separate columns.
This way we have multiple time serieses
.. Model trend as a polynomial function – need to tune coefficients.

We use regular linear regression model to find best fit and find coefficients.

Here is a nice description with Python code:
- https://www.youtube.com/watch?v=td9QLuijh9c

YR	QTR	t_qtr	ySales
2009	1	1	50147
2009	2	2	49325
2009	3	3	57048
2009	4	4	76781
2010	1	5	48617
2010	2	6	50898
2010	3	7	58517
2010	4	8	77691
2011	1	9	50862
2011	2	10	53028
2011	3	11	58849
2011	4	12	79660
2012	1	13	51640
2012	2	14	54119
2012	3	15	65681
2012	4	16	85175
2013	1	17	56405
2013	2	18	60031
2013	3	19	71486
2013	4	20	92183
2014	1	21	60800
2014	2	22	64900
2014	3	23	76997
2014	4	24	103337
2015	1	25	

QtrCateg	t_qtr	t_qtr2	t_qtr3	ySales
qtr1	1	1	1	50147
qtr2	2	4	8	49325
qtr3	3	9	27	57048
qtr4	4	16	64	76781
qtr1	5	25	125	48617
qtr2	6	36	216	50898
qtr3	7	49	343	58517
qtr4	8	64	512	77691
qtr1	9	81	729	50862
qtr2	10	100	1000	53028
qtr3	11	121	1331	58849
qtr4	12	144	1728	79660
qtr1	13	169	2197	51640
qtr2	14	196	2744	54119
qtr3	15	225	3375	65681
qtr4	16	256	4096	85175
qtr1	17	289	4913	56405
qtr2	18	324	5832	60031
qtr3	19	361	6859	71486
qtr4	20	400	8000	92183
qtr1	21	441	9261	60800
qtr2	22	484	10648	64900
qtr3	23	529	12167	76997
qtr4	24	576	13824	103337
qtr1	25	625	15625	



Model	Qtr dummies	t_qtr	t_qtr2	t_qtr3	R2	MAE	RMSE
1	X	X	X	X	0.932	3776.7	4322.7
2	X	X	X		0.962	3124.7	3243
3	X	X		X	0.932	3765.9	4315

Regression: Ridge, LASSO, Elastic Net

Multivariate (multiple) Linear Regression Regularization.

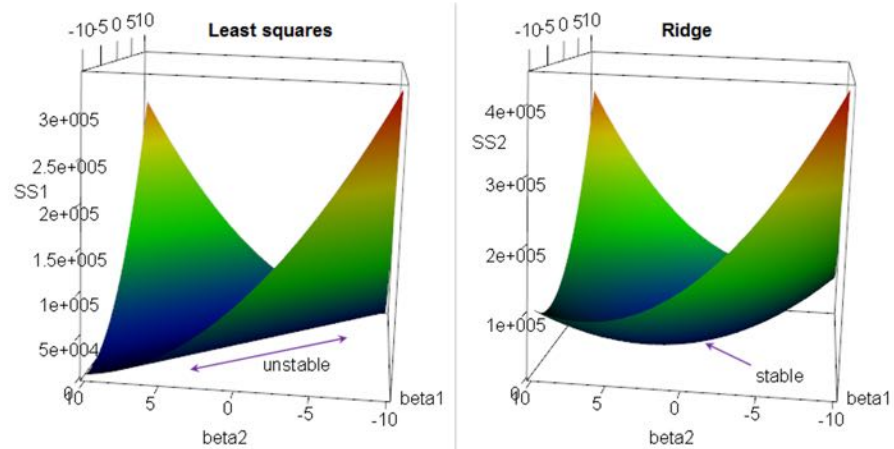
Suppose we have a OLS (Ordinary Linear Squared) regression where two parameters x1 & x2 are not orthogonal, but in fact highly correlated.
Then there may be is infinite number of coefficients which will fit the model.
For example, (x1 - x2), (10*x1 - 10*x2), (100*x1 - 100*x2), etc.
If we are trying to minimize the cost function in the multidimensional space of coefficients, we will get an infinite diagonal groove (ridge) for x1=x2.
Any place at the bottom of this ridge is equally good for us.
So we will get a big variability in possible values of coefficients for x1 & x2.
The solution will be unstable.
We can reduce this variability by adding additional regularization factor like this:

$$\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=1}^{p}\beta_j^2 = \mathrm{RSS} + \lambda\sum_{j=1}^{p}\beta_j^2$$

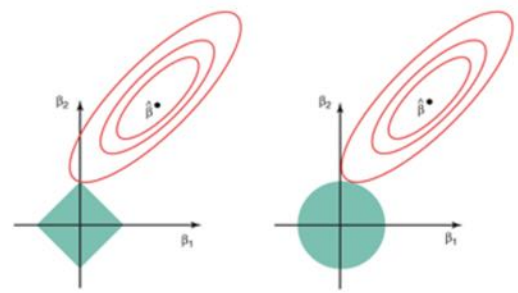
When we minimizing this expression, bigger "lambda" value will cause smaller "beta" values, thus keeping the coefficients "beta" from becoming too large.
This is called "**Ridge Regression**". It changes the infinitely long ridge into a local minimum.

In **Ridge Regression** we add a "**regularization**" term as sum of squares (**L2 norm**).
If instead we use **L1 norm** (sum of abs values) - it is called "LASSO Regression",
where **LASSO = Least Absolute Shrinkage and Selection Operator**.
LASSO favors solutions on the tips of the bounding surface for coefficients, where some of coefficients are zero. Thus it favors "sparse" solutions.

A combination of Ridge Regression and LASSO Regression is called "**Elastic Net**"



Why Ridge Regression is called Ridge ...



Why LASSO favors sparse solutions

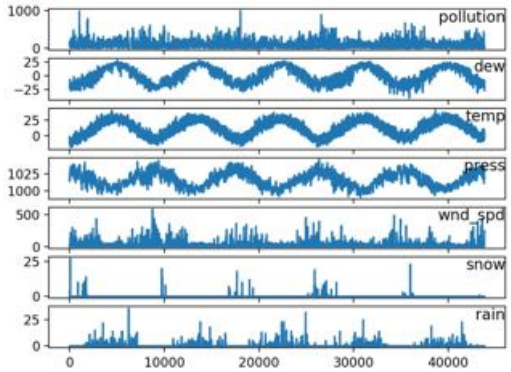
Multivariate Forecasting Using LSTM

Multivariate Time Series Forecasting with LSTMs in Keras (2020) - by Jason Brownlee
- https://machinelearningmastery.com/multivariate-time-series-forecasting-lstms-keras/

Time Series Prediction With Deep Learning in Keras (2020) - by Jason Brownlee
- https://machinelearningmastery.com/time-series-prediction-with-deep-learning-in-python-with-keras/

Neural networks for algorithmic trading. Multivariate time series (2017)
- https://medium.com/machine-learning-world/neural-networks-for-algorithmic-trading-2-1-multivariate-time-series-ab016ce70f57

Energy Demand Time Series Forecasting (2017)
- https://github.com/Azure/MachineLearningSamples-EnergyDemandTimeSeriesForecasting



XGBoost for Time Series Forecasting

"""
XGBoost can be used for time series forecasting, although it requires that the time series dataset be transformed into a supervised learning problem first.

It also requires the use of a specialized technique for evaluating the model called walk-forward validation, as evaluating the model using k-fold cross validation would result in optimistically biased results.
"""
- Jason Brownlee
https://machinelearningmastery.com/

Here are some links with python code:

- https://machinelearningmastery.com/xgboost-for-time-series-forecasting/ -

- https://www.kaggle.com/getting-started/48536 -

- https://towardsdatascience.com/using-gradient-boosting-for-time-series-prediction-tasks-600fac66a5fc

```
# finalize model and make a prediction for monthly births with xgboost
from numpy import asarray
from pandas import read_csv
from pandas import DataFrame
from pandas import concat
from xgboost import XGBRegressor

# transform a time series dataset into a supervised learning dataset
def series_to_supervised(data, n_in=1, n_out=1, dropnan=True):
    n_vars = 1 if type(data) is list else data.shape[1]
    df = DataFrame(data)
    cols = list()
    # input sequence (t-n, ... t-1)
    for i in range(n_in, 0, -1):
        cols.append(df.shift(i))
    # forecast sequence (t, t+1, ... t+n)
    for i in range(0, n_out):
        cols.append(df.shift(-i))
    # put it all together
    agg = concat(cols, axis=1)
    # drop rows with NaN values
    if dropnan:
        agg.dropna(inplace=True)
    return agg.values

# load the dataset
series = read_csv('daily-total-female-births.csv', header=0, index_col=0)
values = series.values
# transform the time series data into supervised learning
train = series_to_supervised(values, n_in=6)
# split into input and output columns
trainX, trainy = train[:, :-1], train[:, -1]
# fit model
model = XGBRegressor(objective='reg:squarederror', n_estimators=1000)
model.fit(trainX, trainy)
# construct an input for a new preduction
row = values[-6:].flatten()
# make a one-step prediction
yhat = model.predict(asarray([row]))
print('Input: %s, Predicted: %.3f' % (row, yhat[0]))
```